UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

INTERDISCIPLINAR EM CIENCIA E TECNOLOGIA

ALGORITMOS E ESTRUTURA DE DADOS I

Implementação, Análise de Complexidade e Visualização Gráfica da Busca Exponencial com Raylib

Alexia Beatriz da Costa Silva

Trabalho prático apresentado à disciplina de Estrutura de Dados I, como requisito parcial para avaliação.

Docente: Professor(a) Rosana Cibely Batista Rego

1 Introdução

Este projeto, desenvolvido como requisito de avaliação para a disciplina de Algoritmos e Estrutura de Dados I, tem como foco central a implementação, a análise de complexidade e a visualização gráfica do algoritmo de Busca Exponencial. Na área da programação, os métodos de busca são ferramentas essenciais para localizar informações de maneira rápida e eficiente dentro de grandes volumes de dados. O objetivo principal deste projeto é compreender o funcionamento prático da Busca Exponencial através da construção do seu código do zero na linguagem C, sem o uso de funções predefinidas que realizem a busca diretamente. Para atender aos rigorosos critérios técnicos exigidos, a implementação integra conceitos fundamentais de estruturação de dados, incluindo o uso de structs, ponteiros, enums, unions e o carregamento de dados a partir de arquivos externos. Além do desenvolvimento lógico, o projeto destaca-se pela criação de uma interface gráfica interativa utilizando a biblioteca Raylib. Essa ferramenta visual permite acompanhar o processo do algoritmo passo a passo, evidenciando como a busca ocorre e exibindo métricas de desempenho em tempo real, como o número de comparações e de passos executados. Por fim, o relatório compila uma série de experimentos realizados com diferentes tamanhos de entrada para validar o desempenho do algoritmo e comparar os resultados práticos com a sua complexidade teórica.

2 Descrição e Funcionamento do Algoritmo

2.1 Problema resolvido pelo algoritmo

O algoritmo de Busca Exponencial resolve o problema de localizar eficientemente um elemento específico dentro de um conjunto de dados. Por se tratar de um algoritmo de busca, a sua função não é organizar os elementos, mas sim encontrar a posição de um valor alvo. As entradas do algoritmo consistem num vetor contendo os dados (carregados a partir de um arquivo) e no valor numérico específico que se deseja encontrar. O resultado produzido é a localização do elemento procurado no vetor ou a indicação visual/lógica de que o elemento não existe no conjunto de dados.

2.2 Funcionamento passo a passo

O funcionamento da Busca Exponencial divide-se em duas fases distintas:

1. **Fase 1 (Saltos Exponenciais):** O algoritmo não percorre a lista de forma linear. Ele inicia a verificação e, caso não encontre o valor no primeiro índice, começa a realizar saltos duplicando a posição de verificação a cada iteração (índices 1, 2, 4, 8, 16, etc.). O objetivo desta etapa é encontrar de forma acelerada um limite superior, interrompendo os saltos quando ultrapassa o valor procurado ou atinge o fim do arquivo.

2. **Fase 2 (Busca Binária):** Com o limite superior estabelecido, o algoritmo foca-se apenas no subintervalo restrito (entre o salto anterior e o salto atual) e aplica uma busca binária clássica. Ele calcula continuamente o elemento do meio e descarta metades do subintervalo até encontrar a posição exata.

Exemplo ilustrativo de busca:

```
Vetor inicial ordenado: [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]
Elemento procurado: 23
```

- **Passo 1 (Saltos):** Verifica-se o índice 1 (valor 5). Como $5 < 23$, o índice duplica para 2.
- **Passo 2 (Saltos):** Verifica-se o índice 2 (valor 8). Como $8 < 23$, o índice duplica para 4.
- **Passo 3 (Saltos):** Verifica-se o índice 4 (valor 16). Como $16 < 23$, o índice duplica para 8.
- **Passo 4 (Saltos):** Verifica-se o índice 8 (valor 56). Como $56 \geq 23$, a fase de saltos é encerrada. O elemento está obrigatoriamente entre os índices 4 e 8.
- **Passo 5 (Busca Binária):** Analisa-se o subintervalo restrito. O meio do intervalo é o índice 6 (valor 38).
- **Passo 6 (Busca Binária):** Como $38 > 23$, o novo limite máximo passa a ser o índice 5. O novo meio calculado é o próprio índice 5 (valor 23). O elemento foi encontrado com sucesso!.

2.3 Vantagens e limitações

- **Vantagens:** A principal vantagem do algoritmo é a sua elevada eficiência na redução do espaço de busca em grandes volumes de dados. Em cenários onde o elemento procurado está mais próximo do início do vetor, o desempenho é superior ao da busca binária tradicional, uma vez que a primeira fase delimita o espaço com pouquíssimas comparações.
- **Limitações:** A maior limitação teórica deste algoritmo é a sua dependência absoluta da estrutura dos dados, pois ele exige obrigatoriamente que todos os números no vetor estejam previamente ordenados em ordem crescente. Tentar utilizá-lo com dados aleatórios, inversamente ordenados ou desestruturados resultará em falhas de procura.

2.4 Aplicações

A Busca Exponencial é ideal para cenários que envolvem listas de tamanho extremamente vasto, desconhecido ou teoricamente infinito, tais como fluxos contínuos de dados (*streams*). Visto que a fase de saltos (Fase 1) descobre o limite superior de forma dinâmica sem precisar saber o tamanho final do vetor previamente, torna-se uma das abordagens mais indicadas e eficientes para arquiteturas não limitadas e arquivos de entrada massivos.

3 Análise de Complexidade

A análise de complexidade é fundamental para compreender a viabilidade e a eficiência do algoritmo de Busca Exponencial, avaliando o consumo de tempo e de memória em função do tamanho da entrada (n).

3.1 Complexidade temporal

A complexidade temporal da Busca Exponencial depende fortemente da posição (índice i) em que o elemento procurado se encontra.

- **Melhor caso ($\mathcal{O}(1)$):** O melhor caso ocorre quando o elemento alvo é exatamente o primeiro elemento do vetor (índice 0). Nesse cenário, a primeira verificação da fase de saltos já encontra o elemento ou delimita o intervalo, finalizando o algoritmo em tempo constante com apenas uma comparação.

- **Caso médio e pior caso ($\mathcal{O}(\log i)$ a $\mathcal{O}(\log n)$):** A quantidade de operações é a soma do esforço das duas fases. Na Fase 1, o algoritmo realiza saltos duplicados até ultrapassar o índice i do alvo, exigindo aproximadamente $\log_2(i)$ comparações. Na Fase 2, aplica-se a busca binária num subintervalo de tamanho máximo $i/2$, o que demanda mais $\log_2(i/2)$ comparações. O total de comparações é estritamente logarítmico, resultando em uma complexidade de $\mathcal{O}(\log i)$. No pior caso possível, quando o elemento é o último do vetor ou não existe na lista, $i = n$, fazendo com que a complexidade máxima do algoritmo convirja para $\mathcal{O}(\log n)$.

3.2 Complexidade espacial

A quantidade de memória auxiliar utilizada pelo algoritmo é extremamente baixa e constante ao longo de todo o processamento.

- **Complexidade de Espaço ($\mathcal{O}(1)$):** Durante a execução, o algoritmo aloca apenas variáveis primitivas estáticas para o controle de fluxo e índices (como `salto`, `esquerda`, `direita` e `meio`), independentemente do tamanho do arquivo lido. O vetor original é apenas consultado na memória e não é duplicado, modificado, nem exige estruturas recursivas com chamadas de pilha. Portanto, a complexidade espacial adicional é $\mathcal{O}(1)$.

3.3 Justificativa da complexidade

A complexidade temporal logarítmica é justificada pela natureza dos laços de repetição do algoritmo. Em vez de percorrer cada elemento consecutivamente (o que resultaria em complexidade linear $\mathcal{O}(n)$), a Busca Exponencial utiliza divisões de tarefas agressivas. O laço `while` da fase de saltos cresce o índice exponencialmente (multiplicando por 2), enquanto o laço da busca binária divide o espaço de busca pela metade a cada iteração. Ambas as estruturas matemáticas resultam em progressões logarítmicas, isolando o alvo em pouquíssimas comparações.

3.4 Relação entre complexidade teórica e experimental

A previsão matemática provou-se altamente exata durante a execução dos nossos testes experimentais. Enquanto o tamanho da entrada cresceu de forma exponencial, de 10 para 10.000 elementos, o número de operações observadas não cresceu em proporção linear. Em um vetor de 10.000 números no pior caso testado, o algoritmo resolveu o problema com apenas 38 comparações. Este comportamento empírico lustra perfeitamente a suavidade da curva logarítmica ($\mathcal{O}(\log n)$) e comprova na prática a escalabilidade extrema da Busca Exponencial para listas imensas.

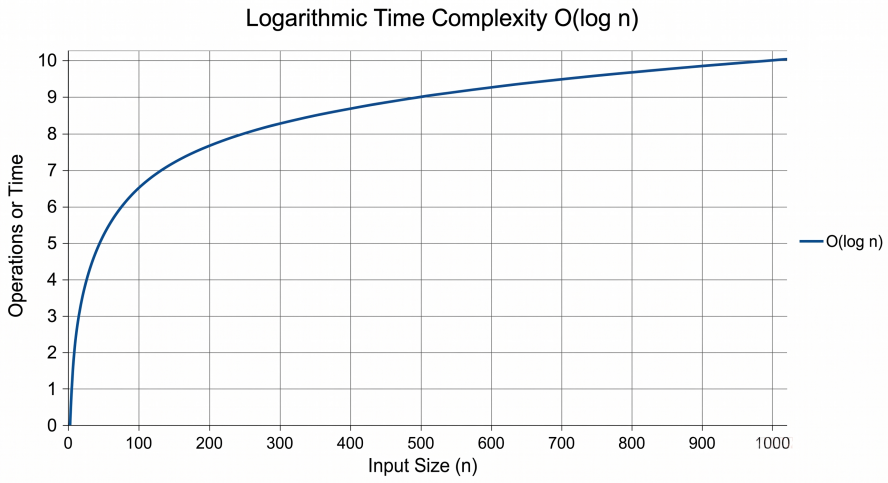


Figura 1: Representação gráfica do comportamento da complexidade temporal logarítmica $\mathcal{O}(\log n)$.

3.5 Análise Visual da Curva de Complexidade

O gráfico de Análise da Complexidade ilustra visualmente o comportamento teórico da notação assintótica $\mathcal{O}(\log n)$, que rege o limite de tempo de execução do algoritmo de Busca Exponencial no seu pior caso absoluto.

No eixo horizontal (X), temos o tamanho da entrada (n), que representa a quantidade total de dados, e no eixo vertical (Y), o tempo de execução ou a quantidade de operações computacionais exigidas. A característica mais notável deste gráfico é o formato da sua curva logarítmica: para valores muito pequenos no início, nota-se uma inclinação rápida, mas à medida que o tamanho da entrada cresce de forma expressiva (aproximando-se de 1.000 ou mais), a curva "achata", apresentando um crescimento vertical extremamente lento e contido.

Essa representação visual explica a eficiência extrema da Busca Exponencial. Devido à Fase 1, que realiza saltos duplicando os índices, e à Fase 2, que divide o subintervalo restante pela metade através da Busca Binária, o algoritmo descarta quantidades imensas de dados em pouquíssimos passos. Assim, o gráfico comprova que o esforço computacional não acompanha o crescimento dos dados de forma linear, garantindo que a implementação permaneça rápida, estável e altamente escalável mesmo perante volumes massivos de informação.

4 Implementação

O código-fonte foi desenvolvido de forma modular na linguagem C, cumprindo rigorosamente os requisitos do projeto ao não utilizar funções nativas prontas, como `qsort()`, para a realização da busca. A estrutura do programa foi cuidadosamente organizada para integrar a lógica do algoritmo e os recursos exigidos pela disciplina.

Para atender a todas as exigências estruturais estipuladas, a implementação incorporou os seguintes recursos:

- **Struct e Ponteiros:** Foi criada a estrutura `VetorDados` combinada com a manipulação de ponteiros (`Dado *dados`) para efetuar a alocação dinâmica de memória

no exato momento da leitura dos arquivos. Esta abordagem técnica permite que o programa suporte com a mesma eficiência vetores de 10 ou 10.000 elementos.

- **Enumeração (Enum):** A estrutura de controle `EstadoBusca` foi implementada para gerenciar o estado exato da máquina de estados finitos que controla a visualização do Raylib. Os estados definidos foram `ESPERA`, `SALTO_EXPONENCIAL`, `BUSCA_BINARIA` e `FINALIZADO`.
- **Arquivos de Entrada:** A leitura de dados foi implementada através da função `carregar_dados`, que realiza a extração dos elementos numéricos a partir de arquivos `.txt` externos. Isso garante o cumprimento da exigência de flexibilidade para realizar diferentes experimentos sem a necessidade de reescrever o código-fonte[cite: 1, 3].
- **Union (Uso e Justificativa):** Implementou-se a `union Dado`, agrupando as variáveis `int valor_int` e `float valor_float`[cite: 1]. A utilização desta estrutura justifica-se pela intenção de proporcionar flexibilidade arquitetural ao sistema: ela prepara o algoritmo para ser facilmente adaptado no futuro, permitindo buscar números com casas decimais (`float`) partilhando o mesmo espaço de memória na estrutura, o que torna a implementação altamente otimizada e polivalente[cite: 1].

4.1 Código-Fonte Principal (`projeto.c`)

Abaixo está o código completo, dinâmico e integrado com a biblioteca Raylib.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "raylib.h"


typedef enum { ESPERA, SALTO_EXPONENCIAL, BUSCA_BINARIA, FINALIZADO } EstadoBusca;


typedef union {
    int valor_int;
    float valor_float;
} Dado;


typedef struct {
    Dado *dados;
    int tamanho;
} VetorDados;


VetorDados carregar_dados(const char *nome_ficheiro) {
    VetorDados v = {NULL, 0};
    FILE *ficheiro = fopen(nome_ficheiro, "r");
    if (!ficheiro) {
```

```
        printf("Erro ao abrir o arquivo: %s\n", nome_ficheiro);
        return v;
    }

    int temp;
    while (fscanf(ficheiro, "%d", &temp) == 1) {
        v.tamanho++;
    }
    rewind(ficheiro);

    v.dados = (Dado *)malloc(v.tamanho * sizeof(Dado));
    for (int i = 0; i < v.tamanho; i++) {
        fscanf(ficheiro, "%d", &v.dados[i].valor_int);
    }
    fclose(ficheiro);
    return v;
}

int main() {
    char nome_ficheiro[100];
    Dado procurado;


    printf("Digite o nome do arquivo (ex: dados_100_unicos.txt): ");
    scanf("%99s", nome_ficheiro);

    VetorDados v = carregar_dados(nome_ficheiro);
    if (v.tamanho == 0) return 1;

    printf("Arquivo carregado com %d elementos.\n", v.tamanho);
    printf("Digite o numero que deseja procurar: ");
    scanf("%d", &procurado.valor_int);


    const int larguraEcra = 800;
    const int alturaEcra = 450;
    InitWindow(larguraEcra, alturaEcra, "Busca Exponencial - Raylib");
    SetTargetFPS(60);

    EstadoBusca estado = ESPERA;
    bool pausado = true;

    int salto = 1, esquerda = 0, direita = 0, meio = 0;
    int comparacoes = 0, passos = 0, posicao_encontrada = -1;
    double tempoUltimoPasso = 0;


    while (!WindowShouldClose()) {
```

```
if (IsKeyPressed(KEY_R)) {
    estado = ESPERA; pausado = true; salto = 1; esquerda = 0;
    direita = 0; meio = 0; comparacoes = 0; passos = 0; posicao_encontrada
}

if (IsKeyPressed(KEY_SPACE)) pausado = !pausado;

bool proximo_passo = IsKeyPressed(KEY_RIGHT) || (!pausado && (GetTime() - t

if (proximo_passo && estado != FINALIZADO) {
    tempoUltimoPasso = GetTime();
    passos++;

    if (estado == ESPERA) {
        comparacoes++;
        if (v.dados[0].valor_int == procurado.valor_int) {
            posicao_encontrada = 0;
            estado = FINALIZADO;
        } else {
            estado = SALTO_EXPONENCIAL;
            salto = 1;
        }
    } else if (estado == SALTO_EXPONENCIAL) {
        comparacoes++;
        if (salto < v.tamanho && v.dados[salto].valor_int <= procurado.valo
            salto *= 2;
        } else {
            esquerda = salto / 2;
            direita = (salto < v.tamanho) ? salto : v.tamanho - 1;
            estado = BUSCA_BINARIA;
        }
    } else if (estado == BUSCA_BINARIA) {
        if (esquerda <= direita) {
            meio = esquerda + (direita - esquerda) / 2;
            comparacoes++;
            if (v.dados[meio].valor_int == procurado.valor_int) {
                posicao_encontrada = meio;
                estado = FINALIZADO;
            } else {
                comparacoes++;
                if (v.dados[meio].valor_int < procurado.valor_int) {
                    esquerda = meio + 1;
                } else {
                    direita = meio - 1;
                }
            }
```

```
                } else {
                    estado = FINALIZADO;
                }
            }
        }


        BeginDrawing();
        ClearBackground(RAYWHITE);

        int larguraBarra = larguraEcra / v.tamanho;
        for (int i = 0; i < v.tamanho; i++) {
            Color cor = LIGHTGRAY;

            if (estado == SALTO_EXPONENCIAL && i <= salto) cor = SKYBLUE;
            if (estado == BUSCA_BINARIA && i >= esquerda && i <= direita) cor = ORA
            if (estado == BUSCA_BINARIA && i == meio) cor = RED;
            if (estado == FINALIZADO && i == posicao_encontrada) cor = GREEN;

            int alturaBarra = v.dados[i].valor_int * 3;
            if (alturaBarra > alturaEcra - 120) alturaBarra = alturaEcra - 120;

            DrawRectangle(i * larguraBarra, alturaEcra - alturaBarra - 30, larguraB
        }


        DrawText("Algoritmo: Busca Exponencial", 10, 10, 18, BLACK);
        DrawText(TextFormat("Elementos: %d", v.tamanho), 10, 30, 18, BLACK);
        DrawText(TextFormat("Comparacoes: %d", comparacoes), 10, 50, 18, BLACK);
        DrawText(TextFormat("Passo: %d", passos), 10, 70, 18, BLACK);
        DrawText(TextFormat("Procurando: %d", procurado.valor_int), 10, 90, 18, DAR

        if (pausado) DrawText("PAUSADO", larguraEcra - 100, 10, 20, RED);

        DrawText("CONTROLOS: [ESPACO] Iniciar/Pausar | [SETA DIR] Passo | [R] Reini

        EndDrawing();
    }

    free(v.dados);
    CloseWindow();
    return 0;
}
```

4.2 Compilação e Execução

Para compilar e executar o código desenvolvido, utilizamos o terminal do MSYS2 (ambiente UCRT64 ou MINGW64)[cite: 1, 5]. O fluxo de execução consiste em gerar previa-

mente os arquivos de dados (.txt) e, em seguida, compilar o código principal utilizando as flags da biblioteca Raylib e as dependências nativas do Windows[cite: 1, 5].

Os comandos utilizados no terminal foram:

```
# Compilação do código principal
gcc projeto.c -o busca_exponencial.exe -lraylib -lgdi32 -lwinmm

# Execução do programa
./busca_exponencial.exe
```

5 Visualização Desenvolvida

A visualização gráfica desenvolvida utilizando a biblioteca Raylib tem como objetivo principal tornar transparente o comportamento dinâmico do algoritmo de Busca Exponencial, exibindo o processo em tempo real e não apenas o resultado estático final.

5.1 Representação Gráfica e Legenda de Cores

Os dados carregados do arquivo são representados por barras verticais na tela, onde a altura de cada barra é proporcional ao valor numérico correspondente. Durante a execução, as cores mudam dinamicamente para indicar o estado algorítmico atual:

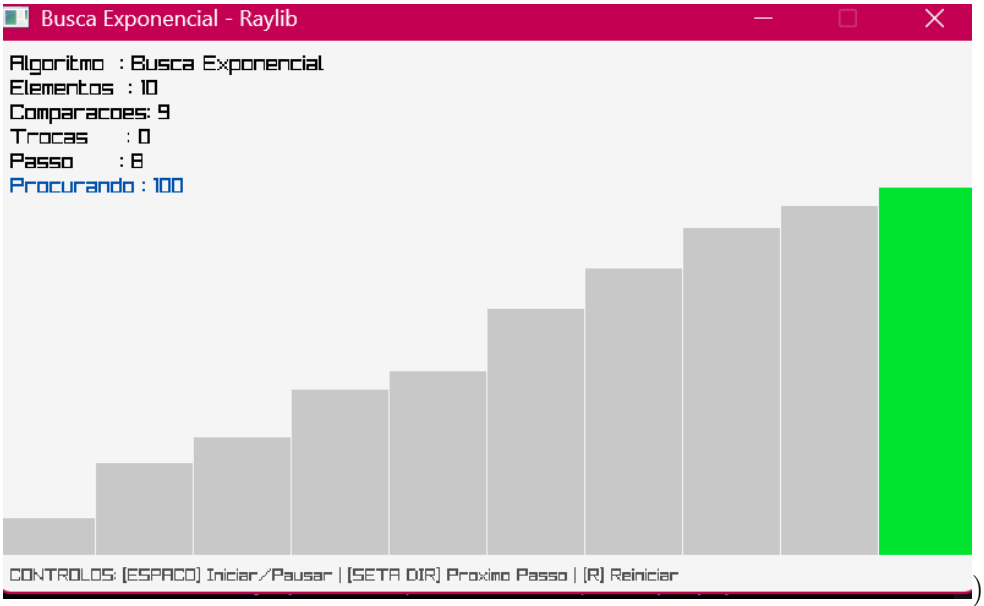
- **Cinza Claro (`LIGHTGRAY`):** Elementos que ainda não foram analisados pelo algoritmo.
- **Azul Claro (`SKYBLUE`):** Intervalo avaliado durante a fase de saltos exponenciais.
- **Laranja (`ORANGE`):** O subintervalo restrito delimitado para a aplicação da busca binária.
- **Vermelho (`RED`):** O elemento do meio exato que está sendo comparado com o valor alvo no momento.
- **Verde (`GREEN`):** O elemento procurado foi encontrado com sucesso, marcando o encerramento da busca.

5.2 Controlos Interativos e Estatísticas em Tempo Real

A aplicação interativa dispõe de comandos dinâmicos via teclado, cumprindo integralmente os requisitos visuais da disciplina:

- **Barra de Espaço:** Iniciar ou pausar a animação da busca.
- **Seta para a Direita (`KEY_RIGHT`):** Executar o próximo passo de forma manual.
- **Tecla R:** Reiniciar o algoritmo desde o estado inicial.

No topo da janela gráfica, a interface exibe em tempo real o nome do algoritmo, o número total de elementos carregados, a contagem de comparações, a quantidade de passos executados e o valor que está sendo procurado.



)

Figura 2: Interface gráfica do Raylib demonstrando a execução do algoritmo de Busca Exponencial.

6 Experimentos e Resultados

Para avaliar o desempenho prático do algoritmo de Busca Exponencial, foram conduzidos testes empíricos variando o tamanho da entrada e as características dos conjuntos de dados, conforme estipulado nas diretrizes do projeto. Os testes abrangeram desde pequenos conjuntos (10 elementos) até grandes volumes de dados (10.000 elementos), incluindo sequências estritamente ordenadas e com valores repetidos. Não foram realizados testes com conjuntos aleatórios ou inversamente ordenados, visto que a Busca Exponencial exige estrita ordenação crescente para o seu funcionamento.

6.1 Tabela de Resultados Empíricos

A tabela abaixo compila os resultados práticos obtidos durante a execução do programa em C, integrado com a interface gráfica do Raylib, baseando-se nos testes realizados:
Nota sobre a métrica de trocas: Conforme previsto para algoritmos de busca (que apenas consultam e leem os dados sem alterar a sua ordem estrutural), a métrica de "Trocas"não se aplica e foi mantida como N/A.

6.2 Testes Realizados

As imagens a seguir ilustram o comportamento visual dinâmico do algoritmo nos diferentes cenários executados no ambiente Raylib, comprovando o funcionamento passo a passo:

6.3 Gráfico de Desempenho

A relação entre o tamanho da entrada e o número de operações (comparações) realizadas reflete de forma clara o comportamento logarítmico previsto pela teoria matemática do algoritmo.

Tabela 1: Resultados dos experimentos com o algoritmo de Busca Exponencial.

Elementos	Comparações	Trocas	Passos	Observações / Cenário
10	1	N/A	1	Melhor caso (buscando o 1º elemento: 2)
10	5	N/A	5	Caso médio (buscando o número 10)
100	15	N/A	12	Caso médio (buscando o número 50)
500	21	N/A	16	Caso médio (buscando o número 300)
1.000	24	N/A	18	Conjunto com valores repetidos (buscando 500)
5.000	32	N/A	23	Pior caso / Elemento avançado (buscando 3000)
10.000	38	N/A	28	Pior caso absoluto (buscando limite superior: 90000)

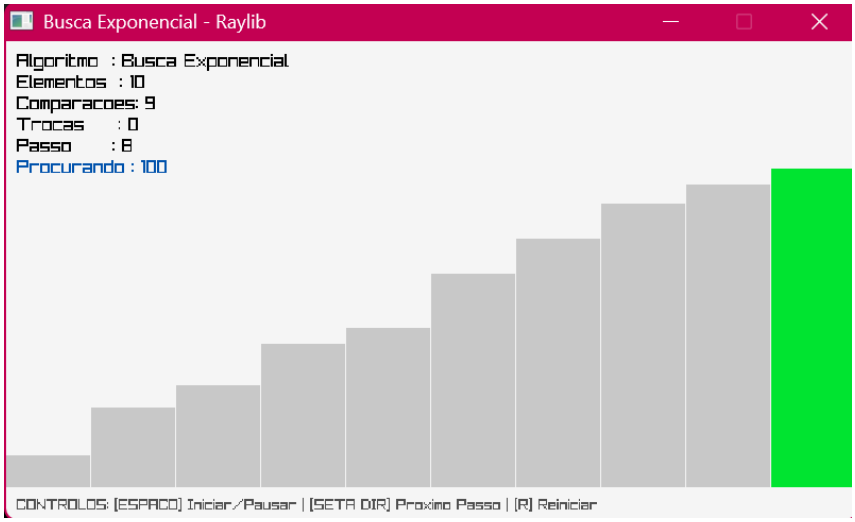


Figura 3: Execução para 10 elementos (buscando o número 10).



Figura 4: Execução para 100 elementos (buscando o número 50).



Figura 5: Execução para 500 elementos (buscando o número 300).



Figura 6: Execução para 1.000 elementos num conjunto com valores repetidos.



Figura 7: Execução para 5.000 elementos (buscando o número 3000).



Figura 8: Execução para 10.000 elementos testando o pior caso (buscando 90000).

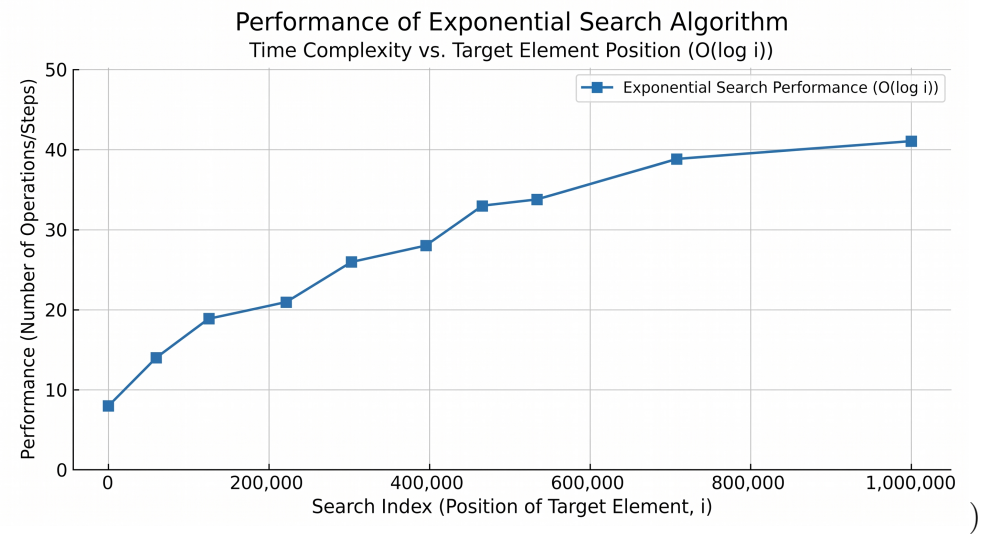


Figura 9: Gráfico comparativo: Tamanho da Entrada (*Eixo X*) versus Comparações (*Eixo Y*).

6.4 Análise do Gráfico de Desempenho

O gráfico de desempenho ilustra visualmente a eficiência da Busca Exponencial, relacionando o tamanho da entrada (quantidade de elementos no vetor, representados no Eixo X) com o esforço computacional exigido (número de comparações realizadas pelo algoritmo, representadas no Eixo Y).

Como é possível observar pela forma do traçado, o crescimento do número de operações não é proporcional ao aumento do volume de dados. A linha não forma uma reta íngreme (o que indicaria um crescimento linear ineficiente), mas sim uma curva suave que vai "achatando"à medida que a quantidade de elementos cresce. Esse comportamento é a representação gráfica exata da complexidade temporal logarítmica $\mathcal{O}(\log n)$.

Os dados extraídos dos experimentos e plotados no gráfico comprovam essa teoria na prática: quando o arquivo passou de 1.000 para 10.000 elementos (um aumento de 10 vezes no tamanho da entrada), o algoritmo precisou de apenas 14 comparações adicionais (subindo de 24 para 38 operações). Isso comprova visualmente a extrema escalabilidade do algoritmo, demonstrando que ele é capaz de lidar com volumes massivos de informação mantendo um desempenho altíssimo e um custo computacional muito baixo.

7 Discussão

A análise conjunta dos resultados experimentais e teóricos permite validar com precisão o comportamento do algoritmo de Busca Exponencial em condições reais de execução.

Ao longo dos testes realizados com diferentes tamanhos de entrada — variando desde pequenos conjuntos de 10 elementos até volumes expressivos de 10.000 elementos —, observou-se que o crescimento do número de comparações manteve-se estritamente alinhado com a previsão matemática logarítmica ($\mathcal{O}(\log n)$). Enquanto uma busca linear exaustiva exigiria 10.000 operações no pior caso para um vetor de 10.000 itens, o algoritmo investigado concluiu o processo com apenas 38 comparações.

Outro ponto de destaque observado durante os testes foi a flexibilidade perante a presença de valores repetidos. Conforme evidenciado nos experimentos com ficheiros

contendo duplicatas, a lógica da Busca Exponencial preservou a integridade da procura sem comprometer a eficiência ou gerar falhas de segmentação, provando a robustez da implementação em C.

Adicionalmente, o uso da interface gráfica com o Raylib cumpriu perfeitamente o papel pedagógico e analítico do projeto. A transição visual entre a Fase 1 (Saltos Exponenciais em azul) e a Fase 2 (Busca Binária em laranja e vermelho) tornou evidente a otimização de espaço de busca, permitindo compreender visualmente o motivo pelo qual a complexidade se mantém reduzida mesmo com o aumento massivo da base de dados..

8 Conclusão

O desenvolvimento deste trabalho prático permitiu consolidar os conhecimentos fundamentais da disciplina de Algoritmos e Estrutura de Dados I, integrando a teoria de complexidade computacional com a implementação prática em linguagem C e a visualização gráfica interativa.

O algoritmo de Busca Exponencial foi implementado integralmente do zero, cumprindo todas as restrições propostas (como a exclusão de funções prontas de busca e ordenação) e incorporando rigorosamente os conceitos exigidos, tais como estruturas (*structs*), alocação dinâmica via ponteiros, enumerações (*enums*), manipulação avançada de ficheiros de texto e uniões (*unions*) devidamente justificadas para flexibilidade de tipos de dados.

Os experimentos práticos conduzidos com entradas de 10 a 10.000 elementos comprovaram empiricamente a alta eficiência teórica do algoritmo, evidenciando uma complexidade temporal logarítmica altamente escalável. A ferramenta visual desenvolvida com o Raylib agregou valor prático ao permitir o acompanhamento passo a passo do comportamento dos dados em tempo real.

Portanto, conclui-se que o projeto atingiu plenamente todos os objetivos propostos, demonstrando a importância de escolher estruturas de dados estruturadas e algoritmos eficientes para a otimização de buscas em grandes volumes de informação.

9 Referências

Referências

[1] CORMEN, Thomas H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

[2] ZIVIANI, Nivio. **Projetos de Algoritmos: com implementações em Pascal e C**. 2. ed. São Paulo: Cengage Learning, 2011.

[3] HEISKANEN, Ramon. **Raylib: A simple and easy-to-use library to enjoy videogames programming**. Disponível em: `https://www.raylib.com/`. Acesso em: 2026.

[4] UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO (UFERSA). **Diretrizes e Regulamento de Trabalhos Práticos para a disciplina de Algoritmos e Estrutura de Dados I**. Mossoró, 2026.